Iridis & VLC GPU Compute

Jonathon Hare / 5-10-2025

VLC GPU Servers

- geoffrey.ecs.soton.ac.uk & yoshua.ecs.soton.ac.uk
  - Both have: 4x RTX2080Ti, 16 cores, 132GB RAM
- minerva.ecs.soton.ac.uk
  - 4x GTX1080Ti, 16 cores, 128GB RAM
- athena.ecs.soton.ac.uk
  - 1x Titan X (Pascal), 3x GTX1080Ti, 16 cores, 128GB RAM

- Direct ssh access.
- Hints (needs updating): http://bit.ly/2rhCVz8

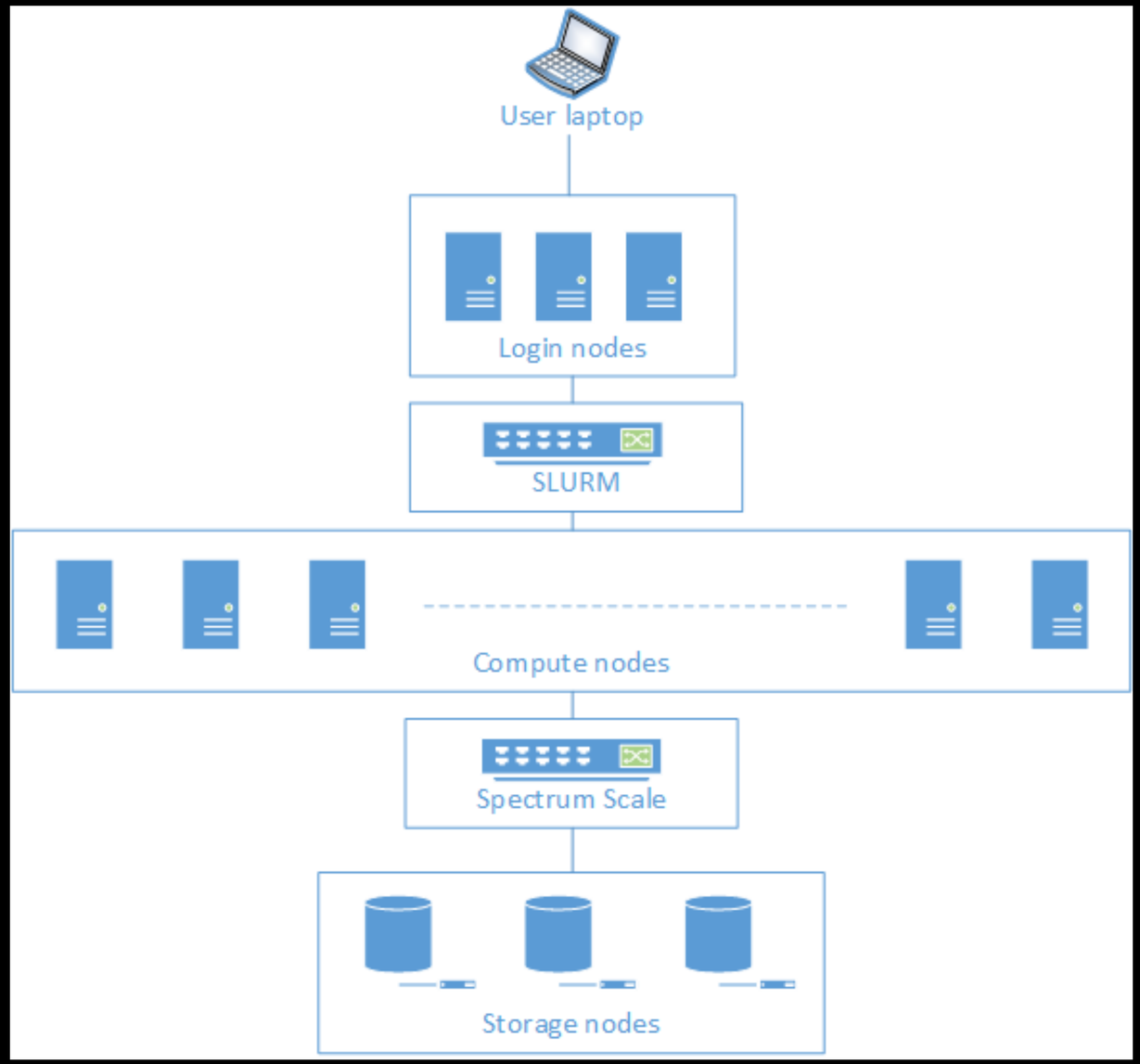
Iridis

- University supercomputer cluster
  - You have access to 3 different variants
    - Iridis 5 (nodes with v100 16GB, GTX1080Ti & the ECS-alpha partition with RTX8000s)
    - Iridis X (Heterogeneous GPU compute: L4s to H200s, including ECS SWARM nodes with L4s, quad A100s and 8-way A100s)
    - *Iridis 6 (zero GPUs; many many CPUs for running parallel scientific workloads using e.g. MPI)*
  - Iridis 7 is coming (plans for it to be in the top-500)

Iridis Storage

- Iridis X and Iridis 6 have a common filesystem (this will extend to Iridis 7 and beyond)
- Iridis 5 uses an older storage platform
- You have separate home directories and scratch directories on both and need to manually copy/sync data between them
- Many small files are your enemy… learn to use hdf5

Iridis Architecture



Iridis 5 Architecture

- Login nodes:
  - iridis5_a.soton.ac.uk
  - iridis5_b.soton.ac.uk
  - iridis5_c.soton.ac.uk

  - /home/<username> - home directory (110GB / 160000 inodes) (backed up)
  - /scratch/<username> - temp storage (1500GB / 500000 inodes)

Iridis X Architecture

- Login nodes:
  - loginx001.iridis.soton.ac.uk (4x L4 GPUs)
  - loginx002.iridis.soton.ac.uk
  - loginx003.iridis.soton.ac.uk (4x L4 GPUs)

  - /home/<username> - home directory (110GB / 204800 inodes) (backed up)
  - /scratch/<username> - temp storage (1500GB / 512000 inodes)
  - /iridisfs/vlcgroup/ - 80ish TB ;)

Basics

- Compute nodes don’t have internet access
  - You can run tensorboard on a login node and connect to it within the Southampton network
  - You can ssh tunnel Jupyter Notebooks (but that’s maybe not the best use of Iridis)
  - You can still use WandB in *offline mode*
    - https://docs.wandb.ai/support/run_wandb_offline/
- Don’t run long/intense jobs on the login node!
- Put your python envs on /scratch as they use lots of inodes
- Use `myquota`

Modules

- Iridis nodes use a system called `lmod` to manage centrally installed software
  - `module av # list all available modules`
  - `module list # list all loaded modules`

  - `module load conda/py3-latest # latest conda on Iridis 5`
  - `module load conda/python3 # latest conda on Iridis X`
  - `module load gcc # load a C compiler`

Slurm

- sbatch
  - sbatch <script.sh>
- sinteractive
- sinfo
- squeue
  - squeue --me # list my jobs
- scancel
  - scancel <jobid> # cancel job <jobid>
  - scancel --me # cancel all jobs
- sacctmgr
  - sacctmgr show qos h200 format=name,MaxTRESPU%50

Slurm Partitions (Iridis 5)

- largejobs
- batch
- gpu
- gtx1080
- lyceum
- serial
- highmem
- relgroup
- worldpop
- hydrology
- veils
- ngcp
- interactive_practical
- orangevm
- niceorange
- ecsstaff
- ecsstudents
- *ecsall*
- *scavenger*
- enginframe

Slurm Partitions (Iridis X)

- a100
- a100_dev
- amd
- amd_dev
- amd_serial
- amd_student
- dual_h200
- ecsstudents_l4
- l4
- l40
- maths_a100
- mi300x
- quad_h200
- *scavenger_4a100*
- *scavenger_8h100*
- *scavenger_l4*
- *scavenger_mathsa100*
- swarm_a100
- swarm_h100
- swarm_l4

sbatch

- Used to launch shell scripts which define a “job” to run on compute nodes
  - You can pass additional arguments to the script:
    - `sbatch launch.sh --config path/to/config.yaml`
    - inside the script use `$@`
  - Job arrays let you launch multiple jobs with a different parameter set
    - https://blog.ronin.cloud/slurm-job-arrays/
    - Useful for multiple runs with different seeds, different configs, …

Anatomy of an sbatch script

single-gpu / single node example

```
#!/bin/bash -l
#SBATCH -p swarm_a100,swarm_h100,dual_h200,quad_h200
#SBATCH --mem=64G
#SBATCH --gres=gpu:1
#SBATCH --nodes=1
#SBATCH -c 10
#SBATCH --time=24:00:00

echo "$@"

module load conda/python3
source activate feature-importance

python -m gaussians.cli.tools.finetune "$@"
```

Anatomy of an sbatch script

multi-gpu / single node example

```
#!/bin/bash -l
#SBATCH -p quad_h200,dual_h200,swarm_h100,swarm_a100
#SBATCH --mem=386G
#SBATCH --gres=gpu:2
#SBATCH --nodes=1
#SBATCH --cpus-per-gpu=10
#SBATCH --time=60:00:00

export OMP_NUM_THREADS=1

# Set the RDZV_HOST variable to the hostname
export RDZV_HOST=$(hostname)

# Generate a reasonably unique number for the communication port.
export RDZV_PORT=$(expr 10000 + $(echo -n $SLURM_JOBID | tail -c 4))

module load conda/python3
source activate feature-importance

torchrun --nproc_per_node=2 --rdzv-endpoint="$RDZV_HOST:$RDZV_PORT" train.py \
         --model convnext_tiny --batch-size 512 --opt adamw --lr 1e-3 --lr-scheduler cosineannealinglr \
         --lr-warmup-epochs 5 --lr-warmup-method linear --auto-augment ta_wide --epochs 600 --random-erase 0.1 \
         --label-smoothing 0.1 --mixup-alpha 0.2 --cutmix-alpha 1.0 --weight-decay 0.05 --norm-weight-decay 0.0 \
         --train-crop-size 176 --model-ema --val-resize-size 232 --ra-sampler --ra-reps 4 \
         --dog.rotation=false --dog.offset=false --dog.size_map.7=25 --output-dir=./models/convnext_tiny_norot_nooffset_noscalewd_25 \
         --dog-scale-weight-decay=0.0 --resume=./models/convnext_tiny_norot_nooffset_noscalewd_25/checkpoint.pth
```

Anatomy of an sbatch script

multi-gpu / multi node example

```
#!/bin/bash -l
#SBATCH -p swarm_h100,swarm_a100
#SBATCH --mem=386G
#SBATCH --gres=gpu:8
#SBATCH --nodes=2
#SBATCH --cpus-per-gpu=10
#SBATCH --time=60:00:00

export OMP_NUM_THREADS=1

# Set the RDZV_HOST variable to the hostname
export RDZV_HOST=$(hostname)

# Generate a reasonably unique number for the communication port.
export RDZV_PORT=$(expr 10000 + $(echo -n $SLURM_JOBID | tail -c 4))

module load conda/python3
source activate feature-importance

torchrun --nproc_per_node=8 --rdzv-endpoint="$RDZV_HOST:$RDZV_PORT" train.py \
         --model convnext_tiny --batch-size 64 --opt adamw --lr 1e-3 --lr-scheduler cosineannealinglr \
         --lr-warmup-epochs 5 --lr-warmup-method linear --auto-augment ta_wide --epochs 600 --random-erase 0.1 \
         --label-smoothing 0.1 --mixup-alpha 0.2 --cutmix-alpha 1.0 --weight-decay 0.05 --norm-weight-decay 0.0 \
         --train-crop-size 176 --model-ema --val-resize-size 232 --ra-sampler --ra-reps 4 \
         --dog.rotation=false --dog.offset=false --dog.size_map.7=25 --output-dir=./models/convnext_tiny_norot_nooffset_noscalewd_25 \
         --dog-scale-weight-decay=0.0 --resume=./models/convnext_tiny_norot_nooffset_noscalewd_25/checkpoint.pth
```

Anatomy of an sbatch script

Other useful directives

```
#SBATCH --mail-type=ALL
#SBATCH --mail-user=your_email@soton.ac.uk
```

Beyond sbatch scripts

- Sometimes you want to jump back into the queue if you run out of time or get preempted…
  - https://github.com/facebookincubator/submitit

Lets go:
Hello World

- Connect to Iridis 5 or X
- Clone starter code
  - https://github.com/ecs-vlc/iridis101/
- Queue a simple job
  - `sbatch simple-sbatch-example.sh`

Lets go:
Training a neural network

- Setup conda
  - conda create -n "my-pytorch-env" python=3.11
    conda activate my-pytorch-env
    pip install torch torchvision torchaudio
    pip install packaging torchbearer
- Download the dataset
  - python cifar.py
- Queue a real job
  - launch-iridisx.sh
  - Monitor resource utilisation using `nvidia-smi` and `top`